PRISMVIEW COLLECTOR · SELF-STUDY WORKBOOK

M2 셀프 복습 워크북

OKX 1초봉 조회 — 혼자서 다시 만들 수 있을 때까지

범위	기준일	스택
#10 ~ #16 + 브랜치 보호	**2026-11-03**	**Spring Boot 4.1.2 · Java 21**

이 워크북을 쓰는 법

1. **읽지 말고 먼저 막아 두기.** 질문(Q) 아래의 해설은 종이나 손으로 가리고, 점선 칸에 내 답을 먼저 적는다.
2. **해설로 채점.** 틀린 질문 번호에 표시해 두고 이틀 뒤 그 번호만 다시 푼다.
3. **손으로 써 보기(EX)는 Cursor Tab 자동완성을 끄고** 빈 파일에서 시작한다. 정답 칸은 다 쓴 뒤에만 본다.
4. **마지막 장의 재구현 미션**을 AI 없이 끝까지 해 보는 게 이 워크북의 최종 목표다. 막히면 힌트 단계를 하나씩만 연다.

오늘은 대부분 초안을 받아 이해하며 타이핑했다. 이 워크북은 "이해했다"를 "혼자 쓸 수 있다"로 바꾸기 위한 것이다.

CONTENTS

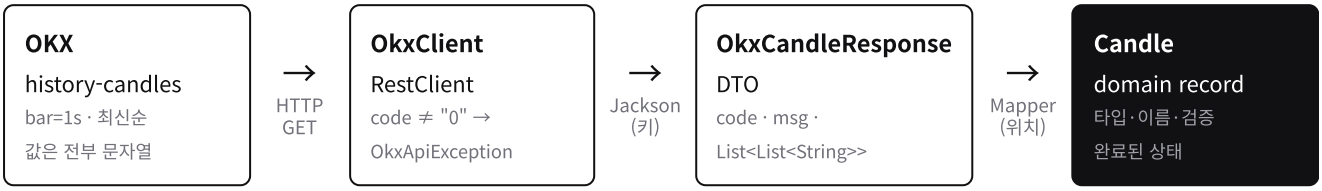
목차

00	전체 그림 — M2에서 만든 것	구조 한 장으로
01	Java 기초 문법 — Python과 비교	record · 타입 · BigDecimal · 비교
02	#10 Candle — 도메인 레코드와 검증	무엇을 막고 무엇을 허용하나
03	#11 DTO와 Jackson 매핑	키 vs 위치
04	#12 OkxClient — Spring과 HTTP	주입 · 설정 · 프로필 · 업무 오류
05	#13 목 서버 테스트	가짜 응답은 진짜처럼
06	#14 Testcontainers	의존을 끊고 증명하기
07	#15 CI와 브랜치 보호	알림에서 강제로
08	#16 1초봉 보관 결정과 ADR	선택지를 숫자로
09	에러 읽기 드릴	오늘 실제로 본 에러 4개
10	재구현 미션 — AI 없이	빈 패키지에서 OkxClient까지
11	자기 점검표 & Python ↔ Java 대응표	설명할 수 있나?

00

전체 그림 — M2에서 만든 것

M2의 목표는 한 문장이다. **OKX에서 1초봉 한 페이지(최대 100개)를 받아, 검증된 `List<Candle>`로 돌려준다.** 나머지(테스트·CI·보호 규칙·ADR)는 그걸 믿을 수 있게 만드는 장치다.



층	클래스 / 파일	한 줄 역할
domain	`Candle`	거래소와 무관한 "1초봉 하나". 생성 순간 검증.
okx	`OkxCandleResponse`	OKX가 보낸 JSON 모양 그대로 (문자열).
okx	`OkxCandleMapper`	행의 위치(인덱스) → 이름과 타입이 붙은 Candle.
okx	`OkxClient`	HTTP 호출 + 업무 오류(code) 확인 + 매퍼 호출.
okx	`OkxApiException`	HTTP 200이어도 code ≠ "0"일 때.
okx	`OkxManualCheckRunner`	`okx-check` 프로필에서만 실제 호출해 로그 출력.
test	`CandleTest`, `OkxCandleMapperTest`, `OkxClientTest`	각 층을 따로 검증. 실제 OKX·로컬 DB 없이 통과.
test	`TestcontainersConfiguration`	테스트마다 일회용 PostgreSQL 17.
repo	`.github/workflows/ci.yml`, Ruleset	PR마다 테스트, 통과·승인 없으면 main 불가.
hub	`docs/adr/0005-...`	1초봉: 최근 3개월 DB, 이후 월별 Parquet.

Q0-1 **위 그림에서 Candle만 검은 상자다. "OKX가 내일 응답 형식을 바꾼다"면 어느 상자들을 고쳐야 하고, 어느 상자는 그대로일까?**

먼저 내 생각 적기

해설

DTO(`OkxCandleResponse`)와 매퍼(`OkxCandleMapper`), 경우에 따라 `OkxClient`까지만 고친다. `Candle`과 앞으로 만들 집계·저장 코드는 그대로다. 이게 "외부 형식과 내부 개념을 분리한다"의 실제 이득이고, Candle을 따로 만든 이유다.

Q0-2 **M2를 "OKX 호출 코드만 있으면 끝"이 아니라 테스트·CI·보호 규칙·ADR까지 포함해 끝낸 이유를 한 문장으로 설명해 보자.**

먼저 내 생각 적기

해설

M3부터는 15만 번 넘게 호출하며 실제 데이터를 쌓는다. 그 전에 "코드가 맞다"를 사람의 기억이 아니라 자동 장치(테스트·CI·보호 규칙)로 보장하고, 데이터를 어떻게 남길지(ADR)를 정해 둬야 백필 도중 방향을 바꾸지 않는다.

01

Java 기초 문법 — Python과 비교

핵심 요약

- Java는 **타입을 반드시** 쓰고, 순서가 Python 힌트와 반대: `long ts` (Python `ts: int`)
- 문장 끝 `;`, 블록은 `{ }`, 파일 맨 위 `package` = 폴더 경로
- `public` 타입 이름 = 파일 이름. 모든 메서드는 클래스 안에.
- 소문자 타입(`long`, `int`, `boolean`)은 **기본형** → null 불가. 대문자(`Long`, `String`, `BigDecimal`)는 **객체** → null 가능
- `if`는 boolean만. 객체 비교는 연산자 대신 메서드(`equals`, `compareTo`)

1-1. record

```
// Python: @dataclass(frozen=True) class Product: name: str; price: Decimal
public record Product(String name, BigDecimal price) {   // 필드 = 소괄호
    public Product {                                      // compact constructor = 검증 = 중괄호
        Objects.requireNonNull(name, "name은 필수");
    }
}
Product p = new Product("사과", new BigDecimal("1200"));  // new 필수
p.price();                                                // 괄호 붙음 (get 접두사 없음)
```

Q1-1 처음 쓴 코드는 `public record Candle(){ long timestamp, BigDecimal open, ... }`였다. 무엇이 문법적으로 틀렸나?

먼저 내 생각 적기

해설

필드 목록은 **소괄호** 안에 써야 한다. 중괄호는 생성자·메서드가 들어가는 몸통이다. `Candle()`처럼 소괄호가 비면 필드가 0개인 record가 되고, 중괄호 안의 `long timestamp,`는 문법 오류가 된다.

Q1-2 compact constructor 안에 `this.open = open;`을 쓰면 어떻게 될까? 그리고 대입 없이 값은 어떻게 저장될까?

먼저 내 생각 적기

해설

컴파일 에러다. compact constructor는 몸통이 끝난 뒤 Java가 각 필드에 **자동으로** 대입한다. 그래서 검증만 쓰면 된다. `@dataclass`가 `__init__`에서 대입하고 `__post_init__`에선 검증만 하는 것과 같다.

Q1-3 **record의 필드는 생성 후 바꿀 수 없다. Candle에 이 성질이 왜 좋을까? 반대로 record가 맞지 않는 대상은?**

먼저 내 생각 적기

해설

1초봉은 지나간 사실의 기록이라 바뀔 이유가 없다. 바꿀 수 없으면 "생성자 검증을 통과한 값이 끝까지 유지된다"를 믿을 수 있다. 반대로 계속 상태가 변하는 것(예: M3의 백필 진행 상황, 레이트 리미터의 남은 토큰)은 record가 맞지 않는다.

1-2. 타입: 기본형 vs 객체

Java	Python	null	Candle에서
`long`	`int`	불가	`ts` — 없을 수 없는 값
`Long`	`int | None`	가능	`after` — "없음 = 최신부터"가 의미 있음
`boolean`	`bool`	불가	`confirm`
`BigDecimal`	`Decimal`	가능	가격·거래량 (기본형 없음 → null 검사 필요)
`String`	`str`	가능	`instId`

Q1-4 **`1793685281000`을 `int`에 담으면? 테스트 코드에서 이 숫자 끝에 `L`을 붙인 이유는?**

먼저 내 생각 적기

해설

`int` 최대값은 2,147,483,647(약 21억)이고 ms 타임스탬프는 약 1.8조다. 넘친다. 숫자를 그냥 쓰면 Java는 int로 읽으려 하므로 `L`을 붙여 long 리터럴임을 알린다. Python `int`는 크기 제한이 없어서 몰랐던 문제다.

Q1-5 **"대문자로 할까, 소문자로 할까"를 정하는 기준을 한 문장으로.**

먼저 내 생각 적기

해설

"null(없음)이 의미 있는 값인가?" 의미가 있으면 객체형(`Long`), 없을 수 없는 값이면 기본형(`long`)으로 해서 잘못된 상태(없는 ts) 자체를 못 만들게 한다.

1-3. BigDecimal과 비교

```
price.signum() < 0                  // price < 0       (signum: -1, 0, 1)
high.compareTo(low) < 0             // high < low      (compareTo: 음수, 0, 양수)
high.compareTo(low) >= 0            // high >= low     → "원하는 부등호를 0 앞에"
a.compareTo(b) == 0                 // 값이 같다 (자릿수 무시)
new BigDecimal("0.1")               // 문자열로 생성 (double 0.1은 부정확)
```

Q1-6 **Python에선 `if 1:`이 동작한다. Java에서 `if (high.compareTo(low))`라고 쓰면?**

먼저 내 생각 적기

해설

컴파일 에러. `compareTo`는 int를 돌려주고, Java의 `if`는 **boolean만** 받는다. 숫자·빈 리스트를 참/거짓으로 취급하는 truthiness가 없어서 항상 `< 0`, `== 0`처럼 비교식을 써야 한다. (오늘 처음 알았던 것)

Q1-7 **`new BigDecimal("1.0").equals(new BigDecimal("1.00"))`의 결과는? 테스트에서 가격 비교에 `assertEquals` 대신 `compareTo`를 쓴 이유는?**

먼저 내 생각 적기

해설

`false`. `equals`는 값과 함께 **자릿수(scale)**까지 비교한다. OKX가 같은 가격을 다른 자릿수로 줄 수 있으니 값 비교는 `compareTo(...) == 0`로 하는 습관을 들인다.

Q1-8 **매퍼에서 `row.get(8) == "1"`로 confirm을 비교하면 어떻게 될까? 왜 `"1".equals(row.get(8))` 순서로 쓸까?**

먼저 내 생각 적기

해설

객체끼리 `==`는 "같은 객체(주소)인가"를 비교해서, JSON에서 파싱한 "1"과 코드의 "1"은 대부분 false다. 값 비교는 `equals`. 리터럴을 앞에 두면 `row.get(8)`이 null이어도 예외 없이 false가 된다. **Python `==` ↔ Java `equals`**, **Python `is` ↔ Java `==`**.

1-4. 메서드 선언 해부

```
private static void requireNonNegative(BigDecimal value, String name) { ... }
// private  : 이 클래스 안에서만          (Python: 이름 앞 _ 관례)
// static   : 객체 없이 호출하는 함수       (Python: 모듈 수준 함수)
// void     : 반환값 없음                  (Python: -> None)
// (타입 이름, 타입 이름) : 매개변수에도 타입 필수
```

> **정정:** 수업 중 "compact constructor에서는 객체가 미완성이라 static이 아니면 부를 수 없다"고 설명했는데, 이건 정확하지 않다. 인스턴스 메서드도 컴파일은 될 수 있다. `static`이 맞는 진짜 이유는 이 함수가 **객체의 상태를 전혀 쓰지 않는 순수 함수**이기 때문이고, 생성자 안에서 아직 채워지지 않은 필드에 기대는 실수를 원천적으로 막아 준다는 점이다.

EX 1-A 빈칸 채우기 — 보지 않고

```
private static ____ requireNonNegative(________ value, ______ name) {
    if (value.______() < 0) {
        _____ ___ IllegalArgumentException(name + "는 음수 불가: " + value);
    }
}
```

정답

`void` · `BigDecimal` · `String` · `signum` · `throw new`

EX 1-B Python → Java 번역

```
if self.high < self.low:
    raise ValueError(f"high < low: high={self.high}, low={self.low}")
```

정답

```
if (high.compareTo(low) < 0) {
    throw new IllegalArgumentException("high < low: high=" + high + ", low=" + low);
}
```

체크: self 없음 · 괄호로 감싼 조건 · compareTo · throw new · 문자열은 + 로 연결

02

#10 Candle — 도메인 레코드와 검증

결정

- 필드 이름은 DB 컬럼에 맞춘다. OKX `volCcyQuote` → `volQuote` (주석으로 남김)
- 막는 것: null · `ts <= 0` · 음수 · `high < low` — "캔들이라 부를 수 없는 값"
- 허용하는 것: 거래량 0 · `high == low` · `confirm=false`
- 품질 규칙(`high ≥ max(open, close)` 등)은 M4에서 "찾아서 기록"

OKX 응답 한 행 읽기

idx	키	의미	예시 (실제 응답)	Candle
0	ts	봉 **시작** 시각, UTC ms	1793685281000	`long ts`
1–4	o h l c	시가·고가·저가·종가	76721.3 …	`BigDecimal`
5	vol	계약 수 (1계약 = 0.01 BTC)	174.25	`vol`
6	volCcy	BTC 수량	1.7425	`volCcy`
7	volCcyQuote	USDT 금액	133686.60868	`volQuote`
8	confirm	"1" 확정 / "0" 진행 중	1	`boolean`
—	(없음)	요청 때 정한 종목	BTC-USDT-SWAP	`String instId`

Q2-1 **OKX가 "계약 수"로 주는 `vol`을 `long`이 아니라 `BigDecimal`로 받은 근거는 무엇이었나? (실제 응답의 숫자로)**

먼저 내 생각 적기

해설

실제 응답에 `0.41`, `174.25`가 있었다. OKX 선물은 계약을 소수 단위로도 거래한다. **이름이 아니라 실제 값을 보고 타입을 정한다.**

Q2-2 검증을 `high > low`(등호 없음)로 썼다면 어떤 데이터에서 문제가 생길까? 이 규칙을 지키는 테스트 이름은?

먼저 내 생각 적기

해설

체결 없는 초는 거래량 0, o=h=l=c=직전 종가다(M1 #6에서 확인, 오늘 응답 첫 행에서도 봄). 그 봉이 전부 예외가 나 한산한 시간대 백필이 멈춘다. 이를 지키는 테스트가 `체결_없는_초는_거래량0_ohlc_동일해도_허용`이다.

Q2-3 "거래량 0이면 o=h=l=c"는 맞다. 그럼 "o=h=l=c면 거래량 0"도 맞을까? #12 실행 로그를 떠올려 보자.

먼저 내 생각 적기

해설

아니다. 최신 봉이 거래량 10.37인데 o=h=l=c였다(그 1초 동안 같은 가격에서만 체결). 규칙은 한쪽 방향만 성립한다. M4 품질 규칙을 만들 때 방향을 헷갈리면 정상 봉을 이상치로 기록하게 된다.

Q2-4 `confirm=false`인 봉을 생성자에서 막으면 #11에서 무슨 일이 생길까?

먼저 내 생각 적기

해설

응답의 첫 행은 대개 진행 중(confirm "0")이라 매핑하자마자 예외가 난다. "저장하지 않는다"는 저장 단계의 규칙이지 캔들의 성립 조건이 아니다.

Q2-5 생성자에서 `high ≥ max(open, close)`까지 막지 않은 이유는? 막았다면 M3 백필에서 어떤 일이 생길 수 있나?

먼저 내 생각 적기

해설

OKX가 이상한 값을 한 줄만 줘도 생성 단계에서 예외 → 그 페이지 전체, 나아가 백필 전체가 멈춘다. 이런 품질 문제는 멈추지 말고 **찾아서 기록**해야 하므로 M4 정합성 단계로 보냈다.

Q2-6 null 검사를 음수 검사보다 먼저 해야 하는 이유는?

먼저 내 생각 적기

해설

null인 `price`에 `price.signum()`을 먼저 부르면 어떤 필드가 문제인지 안 나오는 `NullPointerException`이 난다. `Objects.requireNonNull(x, "x는 필수")`로 먼저 거르면 메시지가 원인을 알려 주고, 이후 검사는 값이 있다고 믿고 진행할 수 있다.

EX 2-A 스펙만 보고 Candle 전체 쓰기 — 15분

새 파일에서 아래 스펙만 보고 작성한다. 오늘 코드 열어 보기 금지.

- 필드 10개: 종목(문자열), 시작 시각(ms), OHLC, 거래량 3종, 확정 여부 — 이름은 DB 컬럼 기준
- null 검사(필요한 것만), ts 양수, 가격·거래량 음수 금지(0 허용), high ≥ low
- 음수 검사는 헬퍼 메서드 하나로

채점 기준

- [ ] 필드가 소괄호 안, 검증이 `public Candle { }` 안
- [ ] `ts`는 `long`, `confirm`은 `boolean` (null 검사 대상 아님)
- [ ] `Objects.requireNonNull` 8개 (String 1 + BigDecimal 7)
- [ ] `signum() < 0`, `compareTo(low) < 0` — 등호 위치 정확
- [ ] `private static void requireNonNegative(BigDecimal value, String name)`
- [ ] `import java.math.BigDecimal;`, `import java.util.Objects;`

EX 2-B 테스트 케이스 설계 — 코드 말고 목록만

Candle 테스트에 꼭 있어야 할 케이스를 5개 이상 적고, 각각 기대하는 예외 타입을 적어 보자.

정답 예시

케이스	기대
정상 값	생성 성공, 값 확인 (`compareTo == 0`)
거래량 0 + o=h=l=c, confirm=false	생성 성공
close = null	`NullPointerException` (requireNonNull이 던짐)
vol = -1	`IllegalArgumentException`
high < low	`IllegalArgumentException`
ts = 0	`IllegalArgumentException`

Python에선 전부 ValueError로 썼지만 Java 코드가 실제로 던지는 타입에 맞춰야 한다.

03

#11 DTO와 Jackson 매핑

세 단어

- **Jackson**: Java의 JSON 라이브러리 = Python `json` + Pydantic의 JSON→객체
- **DTO**: 외부 데이터의 **모양 그대로** 옮긴 객체 (값은 OKX가 준 문자열 그대로)
- **매핑**: DTO → Candle. 문자열→숫자, "1"→true, 이름 붙이기, instId 채우기

```
{ "code": "0", "data": [ ["1793685281000","76721.3", ... ,"1"], ... ], "msg": "" }
  └── 키 있음 ──┘        └──────────── 키 없음, 위치만 있음 ────────────┘
       ↓ Jackson이 키 이름으로 채움            ↓ 매퍼가 인덱스 상수로 이름 붙임
record OkxCandleResponse(String code, String msg, List<List<String>> data)
```

Q3-1 **DTO에 `ts`, `open` … 필드를 만들어 두면 Jackson이 채워 줄까? 왜 DTO가 3필드로 짧은가?**

먼저 내 생각 적기

해설

못 채운다. Jackson은 JSON 객체의 **키 이름**으로 필드를 찾는데, 행 안에는 키가 없고 위치만 있다. 그래서 바깥 키 3개(code/msg/data)만 받고, 의미 붙이기는 매퍼가 한다. **DTO 모양은 내가 정하는 게 아니라 상대 JSON이 정한다.**

Q3-2 **실제 JSON은 code → data → msg 순인데 record는 (code, msg, data)로 선언했다. 문제 될까? 순서가 의미 있는 곳은 어디일까?**

먼저 내 생각 적기

해설

JSON 객체 `{}`의 키 순서는 의미 없다(Jackson은 이름으로 짝 맞춤). 배열 `[]`은 순서가 전부다. 단, Java 코드에서 `new OkxCandleResponse(...)`로 직접 만들 때는 record 선언 순서대로 인자를 넣어야 한다.

Q3-3 **행 길이 검사를 `!= 9`가 아니라 `< 9`로 한 이유는?**

먼저 내 생각 적기

해설

OKX가 행 끝에 값을 하나 추가하면 `!= 9`는 멀쩡한 응답을 전부 거부해 수집이 멈춘다. 쓰는 건 앞 9개뿐이니 "최소 9개"만 보장하면 된다. 모자랄 땐 `IndexOutOfBounds` 대신 무엇이 부족한지 메시지로 알린다.

Q3-4 **매퍼 테스트를 "실제 3행(값 검증)"과 "fixture 100행(순서·간격 검증)" 두 종류로 둔 이유는? 인덱스 6과 7을 바꿔 끼우면 어느 테스트가 잡을까?**

먼저 내 생각 적기

해설

3행은 값을 아니까 **위치별 정확성**을, 100행은 값을 다 모르는 대신 **규모·최신순·1초 간격**을 본다. 6↔7 교체는 3행 테스트만 잡는다. 하나만 있으면 둘 중 한 종류의 버그를 놓친다.

Q3-5 **fixture 100행 테스트의 "1초 간격" 검사가 실패한다면, 매퍼 버그일까?**

먼저 내 생각 적기

해설

그렇지 않을 수 있다. 이 검사는 #6에서 확인한 "빈 초도 채워 준다"는 **데이터 성질**에 기대고 있다. 실패하면 우리가 믿는 데이터 가정이 깨졌다는 신호다. 테스트가 코드뿐 아니라 가정도 문서로 남긴다.

Q3-6 **인터넷 예제의 `import com.fasterxml.jackson.databind.ObjectMapper;`를 붙였더니 빨간 줄이 뜬다. 왜? 그런데 `@JsonIgnoreProperties`는 왜 여전히 `com.fasterxml`인가?**

먼저 내 생각 적기

해설

Boot 4는 Jackson 3를 쓰고 핵심 클래스가 `tools.jackson`으로 옮겨졌다(예: `tools.jackson.databind.json.JsonMapper`). 어노테이션 라이브러리만 버전 간 공유되어 `com.fasterxml.jackson.annotation`에 남았다. **예제를 볼 땐 버전부터** — 오늘 같은 함정이 Testcontainers 2, Actions 버전에서도 반복됐다.

EX 3-A 매퍼의 핵심 한 줄씩 쓰기

문자열 행 `row`에서 각 값을 꺼내는 Java 표현을 적자.

목표	내 답
ts (long)	
close (BigDecimal)	
confirm (boolean, "1"이면 true)	
행 개수 확인	

정답

`Long.parseLong(row.get(TS))` · `new BigDecimal(row.get(CLOSE))` · `"1".equals(row.get(CONFIRM))` · `row.size() < MIN_ROW_SIZE`

EX 3-B 폴더 이름 하나

fixture JSON을 `getClass().getResourceAsStream("/okx/history-candles-1s.json")`로 읽는다. 파일의 정확한 위치를 적고, 폴더 이름을 틀리면 무슨 일이 생기는지 적자.

정답

`src/test/resources/okx/history-candles-1s.json`. 오늘 `resource`로 만들었더니 Gradle이 클래스패스에 안 올렸고, `getResourceAsStream`이 **예외 없이 null**을 돌려줘 엉뚱하게 Jackson에서 터졌다. 원인과 증상의 위치가 다른 대표적인 경우.

04

#12 OkxClient — Spring과 HTTP

```
@Component                                                  // Spring이 만들고 관리 (주입 대상)
public class OkxClient {
    private final RestClient restClient;

    public OkxClient(RestClient.Builder builder,                // Spring이 넣어 줌 (FastAPI Depe
nds)
                     @Value("${okx.base-url}") String baseUrl) { // 설정값 (settings.x)
        this.restClient = builder.baseUrl(baseUrl).build();
    }

    public List<Candle> fetchOnePage(String instId, Long after) {
        OkxCandleResponse response = restClient.get()
            .uri(b -> b.path("/api/v5/market/history-candles")
                .queryParam("instId", instId).queryParam("bar", "1s").queryParam("limit", 100)
                .queryParamIfPresent("after", Optional.ofNullable(after)).build())
            .retrieve()
            .body(OkxCandleResponse.class);

        if (response == null) throw new OkxApiException("EMPTY", "응답 body 없음");
        if (!"0".equals(response.code())) throw new OkxApiException(response.code(), response.ms
g());
        return OkxCandleMapper.toCandles(instId, response);
    }
}
```

Q4-1 **HTTP 상태는 200인데 body가 `{"code":"51001","msg":"Instrument ID does not exist","data":[]}`다. code 확인이 없다면 어떻게 동작하고, 왜 위험한가?**

먼저 내 생각 적기

해설

에러 없이 **빈 리스트**를 돌려준다. M3 백필에선 "이 구간엔 데이터가 없다"로 오해해 조용히 넘어갈 수 있다. OKX는 업무 오류를 HTTP 상태가 아니라 body의 `code`로 알린다. "누락은 에러를 내지 않고 조용히 백테스트를 왜곡한다"는 로드맵 원칙과 같은 함정.

Q4-2 **`RestClient.create()`로 직접 만들지 않고 Spring이 주는 `RestClient.Builder`를 받은 이유 두 가지.**

먼저 내 생각 적기

해설

① Spring이 주는 Builder엔 Jackson 3 변환 설정이 이미 들어 있어 `.body(OkxCandleResponse.class)`가 바로 동작한다. ② #13에서 `MockRestServiceServer`를 이 Builder에 연결해 가짜 응답을 넣을 수 있다.

Q4-3 **설정값을 필드 주입이 아니라 생성자 매개변수로 받은 선택이 #13에서 어떤 이득이 됐나?**

먼저 내 생각 적기

해설

Spring 없이 `new OkxClient(builder, "https://www.okx.com")`로 손으로 만들 수 있다. 그래서 #13 테스트는 Spring 컨텍스트·DB 없이 1초 안에 끝난다.

Q4-4 **`okx.base-url`을 `application-local.yaml`이 아니라 공통 `application.yaml`에 넣은 이유는? `spring:` 아래로 들여 쓰면?**

먼저 내 생각 적기

해설

OKX 주소는 환경마다 같고 비밀도 아니다. 환경마다 달라지는 값(DB 주소·비밀번호)만 프로필로 뺀다. `spring:` 아래에 넣으면 키가 `spring.okx.base-url`이 되어 같은 에러가 난다.

Q4-5 **수동 확인 러너에 `@Profile("okx-check")`가 없다면?**

먼저 내 생각 적기

해설

앱이 뜰 때마다 OKX를 호출한다. 특히 테스트의 `contextLoads`도 앱을 띄우므로 **자동 테스트가 실제 OKX에 의존**하게 된다. 네트워크가 끊기면 실패, CI에서도 매번 호출.

Q4-6 **로그 줄 앞은 `17:38:56+09:00`, Candle 최신 시각은 `08:38:54Z`. 9시간 차이는 버그인가? 수집기 안에선 어떤 시간대로 다뤄야 하나?**

먼저 내 생각 적기

해설

버그가 아니다. `Z`는 UTC, 한국은 UTC+9. OKX ts도 UTC epoch ms라서 **안에서는 끝까지 UTC**, 사람에게 보여 줄 때만 변환한다. 서버 리전이 바뀌어도 값이 흔들리지 않는다.

Q4-7 **FastAPI 서버는 계속 떠 있는데, okx-check로 실행한 이 앱은 로그를 찍고 스스로 꺼졌다. 왜?**

먼저 내 생각 적기

해설

웹 서버(Tomcat)가 없어서 요청을 기다리며 앱을 붙잡는 스레드가 없다. 시작 → 러너 → 할 일 없음 → 종료. Boot 4의 `spring-boot-starter-restclient`가 서버를 포함하지 않는다는 증거이고, 그래서 이슈의 `web-application-type=none`을 넣지 않았다(나중에 점검 UI를 붙일 때 함정이 되므로).

EX 4-A Spring 어노테이션 짝 맞추기

하고 싶은 것	Java/Spring
이 클래스를 Spring이 만들어 주입 대상으로	
설정 파일 값 읽기	
특정 프로필에서만 빈 등록	
앱 시작 직후 한 번 실행	
로그 남기기	

정답

`@Component` · `@Value("${키}")` · `@Profile("이름")` · `implements CommandLineRunner` + `run()` · `LoggerFactory.getLogger(X.class)`, `log.info("{}", v)`

05

#13 목 서버 테스트

```
@BeforeEach
void setUp() {
    RestClient.Builder builder = RestClient.builder();
    server = MockRestServiceServer.bindTo(builder).build();   // build() 전에 연결해야 가로챔
    okxClient = new OkxClient(builder, BASE_URL);             // Spring 없이 직접 생성
}

server.expect(requestTo(startsWith(BASE_URL + PATH)))
      .andExpect(queryParam("bar", "1s"))
      .andRespond(withSuccess(errorBody, MediaType.APPLICATION_JSON));   // HTTP 200 + code=5100
1
```

테스트	가짜 응답	기대	M3에서 쓰임
정상	fixture 100행	100개, 경로·쿼리 일치	—
after	fixture	`after` 쿼리 포함	페이지네이션
업무 오류	**200** + code 51001	`OkxApiException`	재시도 **안 함**
429	HTTP 429	`HttpClientErrorException.TooManyRequests`	재시도 대상
500	HTTP 500	`HttpServerErrorException`	재시도 대상

Q5-1 **업무 오류 테스트에서 가짜 서버가 HTTP 400을 돌려주게 했다면, 테스트는 통과할까? 무엇이 검증되지 않을까?**

먼저 내 생각 적기

해설

RestClient가 HTTP 예외를 먼저 던지므로 `OkxApiException`을 기대하는 테스트는 실패하거나, 예외 타입을 느슨하게 썼다면 통과해 버린다. 어느 쪽이든 내가 만든 **code 확인 코드는 한 번도 실행되지 않는다**. 가짜 응답은 실제 상대(OKX)의 동작을 흉내 내야 의미가 있다.

Q5-2 **"code 확인 블록을 주석 처리하면 업무 오류 테스트만 실패한다"는 실험의 목적은?**

먼저 내 생각 적기

해설

테스트가 정말 그 코드를 지키고 있는지 확인하는 것. 지켜야 할 코드를 망가뜨렸는데 테스트가 통과한다면 그 테스트는 아무것도 검증하지 않는 것이다. "일부러 망가뜨려 보기"는 오늘 Candle(< → <=), 매퍼(6↔7)에도 쓴 방법이다.

Q5-3 429와 500 테스트는 "예외가 난다"만 확인하고 끝난다. 그런데도 지금 만든 이유는?

먼저 내 생각 적기

해설

M3 재시도는 "429와 5xx만 재시도, 나머지는 즉시 실패"로 정할 예정이고, 그 판단 기준이 예외 타입이다. 지금 타입을 테스트로 고정해 두면 기준으로 그대로 쓰고, 나중에 클라이언트를 고쳐 타입이 바뀌면 바로 알 수 있다.

EX 5-A pytest → JUnit 번역

```
def test_code가_0이_아니면_예외(okx_client, mocked):
    mocked.get(URL, json={"code":"51001","msg":"x","data":[]}, status=200)
    with pytest.raises(OkxApiException) as e:
        okx_client.fetch_one_page("BTC-USDT-SWAP", None)
    assert e.value.code == "51001"
```

정답

```
@Test
void code가_0이_아니면_OkxApiException() {
    server.expect(requestTo(startsWith(BASE_URL + PATH)))
          .andRespond(withSuccess("{\"code\":\"51001\",\"msg\":\"x\",\"data\":[]}", Me
diaType.APPLICATION_JSON));

    OkxApiException e = assertThrows(OkxApiException.class,
            () -> okxClient.fetchOnePage("BTC-USDT-SWAP", null));

    assertEquals("51001", e.code());   // 기대값이 앞
}
```

06

#14 Testcontainers

전

`@ActiveProfiles("local")`
application.yaml + application-local.yaml
→ **localhost:5432** (내가 띄운 docker compose)
내 맥 ✅ · CI ❌

후

`@Import(TestcontainersConfiguration.class)`
application.yaml만
→ **@ServiceConnection 컨테이너** (postgres:17)
내 맥 ✅ · CI ✅

```
@TestConfiguration(proxyBeanMethods = false)
public class TestcontainersConfiguration {
    @Bean
    @ServiceConnection                                    // 컨테이너 주소·계정 → datasource 자동 연결
    PostgreSQLContainer postgresContainer() {
        return new PostgreSQLContainer("postgres:17");    // docker-compose와 같은 버전
    }
}
```

Q6-1 **매번 DB를 띄웠다 지우는 이유 세 가지. (Python으로 치면 무엇과 같은가?)**

먼저 내 생각 적기

해설

① 이전 실행 데이터가 남지 않아 테스트끼리 영향 없음 ② 내 맥과 CI의 출발점이 같음 ③ 빈 DB에 Flyway가 V1부터 적용되는지 매번 검증(새 서버 첫 배포와 같은 상황). + 개발 DB(백필 데이터)와 분리. pytest fixture의 생성/정리, `tmp_path`와 같다.

Q6-2 **#14는 새 테스트를 만들지 않았다. 그럼 무엇이 검증되게 된 건가? `contextLoads`가 통과하려면 무엇이 성공해야 하나?**

먼저 내 생각 적기

해설

DB 연결 → Flyway V1 적용 → 빈(OkxClient 등)과 설정 조립. 이게 **어디서든**(CI 포함) 되도록 DB의 출처만 바꾼 것이다. 설정 누락(`okx.base-url` 같은)도 이 테스트가 잡는다.

Q6-3 `@ActiveProfiles("local")`를 그대로 둬도 통과한다. 그런데 왜 뺐나? 반대로 `@Import`를 깜빡하면?

먼저 내 생각 적기

해설

`@ServiceConnection`이 local 설정보다 우선이라 통과는 한다. 하지만 local 프로필에 개발용 설정이 늘어나면 테스트가 몰래 물려받는다. 빼야 "테스트는 local에 의존하지 않는다"가 코드로 보인다. `@Import`를 깜빡하면 컨테이너도 local 값도 없어 DB 연결에서 실패한다 — 두 어노테이션은 한 세트.

Q6-4 확인을 `docker compose down` 상태에서 한 이유는?

먼저 내 생각 적기

해설

켜 둔 채 통과하면 컨테이너를 썼는지 여전히 localhost에 붙었는지 구분할 수 없다. **의존을 끊었다는 건 끊은 상태에서 통과하는 걸로 증명**한다. (#13의 "네트워크 없이 통과"와 같은 방식)

EX 6-A 의존성 범위 고르기

아래 의존성을 `implementation` / `testImplementation`으로 나누고, 이유를 한 줄씩.

`spring-boot-starter-jackson` · `spring-boot-starter-restclient` · `spring-boot-starter-restclient-test` · `spring-boot-testcontainers` · `testcontainers-junit-jupiter` · `testcontainers-postgresql`

정답

implementation: jackson, restclient (앱 실행에 필요) · **testImplementation**: 나머지 4개 (테스트에서만). 테스트 도구를 implementation으로 넣으면 운영 jar에 들어가고, 운영 코드에서 실수로 import해도 컴파일된다. Python의 requirements.txt / requirements-dev.txt 구분.

07

#15 CI와 브랜치 보호

```
on:
  pull_request: { branches: [main] }   # main으로 가는 PR마다
  push:         { branches: [main] }   # 머지 직후에도 한 번 더
jobs:
  test:                                # ← 이 이름이 status check 이름
    runs-on: ubuntu-latest             # Docker 기본 제공 → Testcontainers 동작
    timeout-minutes: 15
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-java@v5
        with: { distribution: temurin, java-version: '21' }
      - uses: gradle/actions/setup-gradle@v6
      - run: ./gradlew test
```

Q7-1 PR에서 통과했는데 왜 main push에서도 또 돌릴까?

먼저 내 생각 적기

해설

PR 통과 후 머지 시점에 main이 다른 PR로 바뀌었다면 **합쳐진 결과**는 검증된 적이 없다. main push에서 한 번 더 돌려야 "main은 항상 통과 상태"를 확인할 수 있다. 오늘 Actions 화면에 CI #1(PR)과 #2(main)가 두 줄로 남았다.

Q7-2 두 번째 실행(1m 6s)이 첫 번째(1m 2s)보다 빠르지 않았다. 캐시가 안 된 걸까?

먼저 내 생각 적기

해설

setup-gradle은 기본적으로 **main에서만 캐시를 저장**하고 PR 브랜치에선 읽기만 한다(PR이 캐시를 오염시키지 않게). #1(PR)은 저장 못 했고 #2(main)가 처음 저장했다. 다음 PR부터 빨라지는지 확인해 볼 것.

Q7-3 로컬 `gradlew`엔 실행 권한(x)이 있는데 CI에서 `Permission denied`가 날 수 있다. 왜? 확인 명령은?

먼저 내 생각 적기

해설

CI는 **Git에 기록된 권한**대로 파일을 받는다. Windows 커밋, 압축 해제, `core.fileMode=false` 등으로 기록이 빠질 수 있다. `git ls-files -s gradlew` → `100755`면 OK. 오늘은 OK였다.

브랜치 보호 (Ruleset)

규칙	막는 것
Enforcement: **Active**	Disabled면 저장만 되고 아무것도 안 막음 (오늘 놓칠 뻔함)
Require PR, 승인 1	main 직접 push, 작성자 혼자 머지 (lgtm-minseok 승인 필요)
Require status check `test`	CI 빨강인 PR 머지
Restrict deletions / Block force pushes	main 삭제, 이력 덮어쓰기
Bypass 비움	소유자도 예외 없음

Q7-4 **main에 빈 커밋 push → `GH013` 거부. 그런데 "승인 1개 필요"는 에러에 안 나왔다. 규칙이 빠진 걸까?**

먼저 내 생각 적기

해설

아니다. 직접 push는 PR이 아니라서 "PR로 하라"에서 이미 막힌다. 승인 수는 PR 안에서만 의미 있는 규칙이다. 다음 PR에서 승인 전 머지 버튼이 막히는지로 확인한다.

Q7-5 **실험 후 `git reset --hard origin/main` 을 안 하면?**

먼저 내 생각 적기

해설

로컬 main에 원격엔 없는 빈 커밋이 남는다. 다음 pull 때 갈라져 병합 커밋이 생기거나, 그 상태에서 딴 브랜치에 실험 커밋이 섞여 PR에 들어간다. 실험이 끝나면 원격과 같은 상태로 되돌린다.

EX 7-A 순서 맞추기

다음을 올바른 순서로: (a) Ruleset에서 `test` 체크 필수 지정 (b) Testcontainers 도입 (c) ci.yml 추가 후 한 번 실행 (d) 로컬 DB 끄고 테스트 통과 확인

정답

b → d → c → a. CI 서버엔 로컬 DB가 없으니 Testcontainers가 먼저이고, 끊긴 걸 확인한 뒤 CI를 붙인다. 체크 이름 `test` 는 CI가 한 번 돌아 GitHub에 등록된 뒤에야 Ruleset에서 고를 수 있다.

08

#16 1초봉 보관 결정과 ADR

규모 (2종목)	행 수	용량 (추정, 인덱스 포함)
하루	2 × 86,400 ≈ 17만	—
3개월	≈ 1,555만	약 2~4 GB
1년	≈ 6,300만	약 10~15 GB → M3 후 실측

선택지	판단
A. DB에 영구	가능(디스크 259GB). 하지만 DB가 계속 커져 백업·VACUUM·운영 부담 증가
B. 저장 안 함	탈락. 3개월 지나면 재집계·다른 초 봉 생성이 영원히 불가
C. 단기 보관 후 삭제	탈락. 기간 ≤ 3개월이면 OKX에 다시 받는 것과 거의 같음
D. 3개월 DB + 월별 Parquet	**채택.** 전 기간 원본 + 가벼운 DB

Q8-1 **로드맵은 C를 추천했었다. C의 보관 기간을 90일 이하로 잡으면 B보다 나은 게 뭘까?**

먼저 내 생각 적기

해설

거의 없다. OKX가 이미 3개월치를 갖고 있으니 그 안의 데이터는 다시 받으면 된다(호출 시간만 아낌). C가 의미 있으려면 기간이 3개월보다 길어야 한다. **선택지마다 "무엇을 얻나"를 숫자로 따진다.**

Q8-2 **디스크가 충분한데 왜 A가 아니라 D인가? ADR에 "디스크가 부족해서"라고 쓰면 왜 틀린가?**

먼저 내 생각 적기

해설

근거가 사실과 다르면 나중에 디스크를 늘렸을 때 "그럼 A로 바꿔도 되나?"를 잘못 판단한다. 진짜 이유는 **DB를 가볍게**(백업·복원 시간, VACUUM·인덱스 부담) 유지하는 것. 지나간 1초봉은 바뀌지 않고 거의 조회하지 않으니 압축 파일로 — 데이터의 "온도"에 따른 분리.

Q8-3 아카이브 시점을 1개월이 아니라 3개월로 한 이유는?

먼저 내 생각 적기

해설

3개월 = OKX가 다시 줄 수 있는 기간. 그 안에서는 누락을 발견하면 OKX에 재요청해 DB에서 고칠 수 있다. 3개월이 지나면 더는 고칠 수 없으니 그때 "확정"하고 아카이브한다. OKX의 제약과 정확히 맞물리는 규칙.

Q8-4 "검증 통과 후에만 DROP" 단계가 없으면 어떤 사고가 날 수 있나? DELETE 대신 파티션 DROP을 쓰는 이유는?

먼저 내 생각 적기

해설

내보내기가 중간에 실패한 파일인데 DROP까지 이어지면 그 달의 1초봉은 DB에도 OKX에도 없다 — B를 탈락시킨 이유가 D 안에서 사고로 재현. DELETE는 죽은 행이 남아 VACUUM 전까지 용량·성능을 잡아먹고, 파티션 DROP은 즉시 끝나고 용량도 바로 돌아온다.

Q8-5 ADR-0003("오래된 행을 지우지 않는다")과 충돌한다. 0003 본문을 고쳐서 맞추면 안 되는 이유는?

먼저 내 생각 적기

해설

ADR은 "그때 무엇을 왜 결정했나"의 기록이다. 고쳐 쓰면 결정의 역사가 사라진다. 옛 ADR은 상태만 "일부 대체됨 → ADR-0005"로 바꾸고 새 ADR이 무엇을 바꿨는지 설명한다. Git 커밋, Flyway V1을 고치지 않고 쌓는 것과 같은 원리. 그리고 범위 명시: 바뀌는 건 1초봉뿐, 30초봉 정책은 그대로.

EX 8-A 계산 연습 — 계산기 없이 어림

M3 백필: 2종목 × 90일 × 86,400초, 한 요청 100개, history-candles 한도 2초당 20회. 한도의 80%로 돌리면 대략 몇 시간?

정답

행 ≈ 1,555만 → 호출 ≈ 15.5만 회. 한도 10회/초의 80% = 8회/초 → 15.5만 ÷ 8 ≈ 1.9만 초 ≈ **약 5.4시간**.

09

에러 읽기 드릴

오늘 실제로 본 에러들이다. **원인과 해결을 먼저 적고** 해설을 본다.

읽는 법

- 컴파일 에러: **파일:줄** → **무엇이** → **^ 위치**. 여러 개면 맨 위부터 (아래는 연쇄일 수 있음)
- 런타임 스택트레이스: 맨 위가 아니라 **맨 아래** `Caused by:` 부터. 위는 경로, 아래가 원인

D-1 컴파일 에러

```
CandleTest.java:9: error: ';' expected
    private static final String INST_ID = "BTC-USDT-SWAP"
                                                        ^
```

먼저 내 생각 적기

해설

9번째 줄 끝 세미콜론 누락. Java는 실행 전 **컴파일 단계에서** 파일 전체를 검사하므로 테스트가 하나도 안 돌고 `compileTestJava FAILED`로 끝난다.

D-2 앱 시작 실패 (긴 스택트레이스 중 두 줄)

```
BeanCreationException: Error creating bean with name 'okxClient' ...
Caused by: PlaceholderResolutionException: Could not resolve placeholder 'okx.base-url' in va
lue "${okx.base-url}"
```

먼저 내 생각 적기

해설

증상은 OkxClient 생성 실패, 원인은 설정값 없음. `application.yaml` 최상위에 `okx.base-url` 추가. Python `os.environ["X"]`의 KeyError와 같다. 앱 시작 2초 만에 실패한 건 좋은 것 — "빨리, 시끄럽게 실패".

D-3 fixture 테스트가 Jackson 쪽 에러로 실패. 코드는 맞는데 파일은 `src/test/resource/okx/…` 에 있다.

먼저 내 생각 적기

해설

Gradle은 `resources`(복수형)만 클래스패스에 올린다 → `getResourceAsStream`이 예외 없이 **null** → 그 null을 받은 Jackson이 엉뚱한 메시지로 실패. 원인과 증상의 위치가 다르다. `Objects.requireNonNull(stream, "fixture 없음: " + path)`로 감싸면 원인 자리에서 터진다.

D-4 push 결과

```
remote: error: GH013: Repository rule violations found for refs/heads/main.
remote: - Changes must be made through a pull request.
remote: - Required status check "test" is expected.
 ! [remote rejected] main -> main (push declined due to repository rule violations)
```

먼저 내 생각 적기

해설

에러가 아니라 **성공**이다. 보호 규칙(PR 필수, test 필수)이 의도대로 막았다. 원격 main은 그대로이니 로컬을 `git reset --hard origin/main`으로 맞춘다. 보호 장치는 "만들었다"가 아니라 "막히는 걸 봤다"로 끝낸다.

EX 9-A 예측해 보기 — 실제로 돌려서 확인

이렇게 바꾸면	무슨 일이? (예측 → 실제)
Candle에서 `< 0`을 `<= 0`으로	
매퍼의 `VOL_CCY = 6`, `VOL_CCY_QUOTE = 7`을 서로 바꿈	
OkxClient의 code 확인 블록 주석 처리	
OkxManualCheckRunner의 `@Profile` 삭제 후 `./gradlew test`	
테스트의 `@Import` 삭제 + 로컬 DB 끔	

예상 답

① 거래량 0 테스트만 실패 ② 샘플 3행 테스트만 실패 ③ 업무 오류 테스트만 실패(빈 리스트) ④ contextLoads가 실제 OKX 호출 — 네트워크 있으면 통과해 버려 문제를 못 봄(그래서 더 위험) ⑤ DB 연결 실패로 contextLoads 실패. **확인 후 반드시 원복.**

10

재구현 미션 — AI 없이

규칙

- 새 브랜치(예: `practice/m2-rebuild`)에서 `domain`, `okx` 패키지를 지우고 시작한다. **머지하지 않는다.**
- Cursor Tab 끄기. Claude·검색 금지. 막히면 아래 **힌트 1단계**만 연다. 그래도 막히면 2단계.
- 각 단계 시간을 적는다. 다음 주에 한 번 더 해서 시간을 비교한다.

#	미션	완료 조건	시간
1	Candle record + 검증	CandleTest 6개 통과	
2	OkxCandleResponse + Mapper	Mapper 테스트 3개 통과	
3	OkxApiException + OkxClient	컴파일 통과	
4	OkxClientTest (5개)	네트워크 끄고 통과	
5	OkxManualCheckRunner	okx-check로 100개 로그	
6	전체	로컬 DB 끄고 `./gradlew test` 통과	

힌트 1단계 (방향만)

#	힌트
1	필드 = 소괄호, 검증 = `public Candle { }`. null 검사는 객체형만. 음수 헬퍼는 `private static`.
2	DTO는 바깥 키 3개. 매퍼는 인덱스 상수 9개 + 최소 길이. confirm은 리터럴 `.equals`.
3	Builder와 base URL을 생성자로. `retrieve().body(...)` 후 null → code 순서로 확인.
4	`bindTo(builder)`는 `new OkxClient(builder, ...)`보다 먼저. 업무 오류는 `withSuccess`(200).
5	`@Component` + `@Profile` + `CommandLineRunner`. 시각은 `Instant.ofEpochMilli`.
6	테스트 클래스에 `@ActiveProfiles`가 다시 들어가 있지 않은지.

힌트 2단계 (자주 막히는 지점)

- `package` 줄과 폴더 경로가 같은가? 메인 클래스 패키지 아래인가?
- Jackson import가 `tools.jackson...`인가? 어노테이션만 `com.fasterxml.jackson.annotation`.
- `application.yaml`의 `okx:`가 들여쓰기 0칸인가?
- fixture 경로: `src/test/resources/okx/` — 복수형 `resources`.

- `BigDecimal` 비교에 `<`나 `equals`를 쓰지 않았나?
- long 리터럴 끝에 `L`을 붙였나? 문장 끝 `;`은?

회고 (미션 후 직접 적기)

질문	내 답
힌트 없이 끝낸 단계	
힌트 2단계까지 연 단계와 이유	
오늘 이해했다고 생각했는데 못 쓴 것	
M3에서 초안 없이 해 볼 이슈	

11

자기 점검표 & Python ↔ Java 대응표

각 항목을 **아무것도 보지 않고 소리 내어 30초 설명**할 수 있으면 체크한다. 못 하면 괄호 안 Q 번호로 돌아간다.

- [ ] record와 compact constructor (Q1-1, 1-2)
- [ ] 기본형 vs 객체형, null 기준 (Q1-5)
- [ ] compareTo / signum / equals의 차이 (Q1-6~1-8)
- [ ] Java `if`에 truthiness가 없음 (Q1-6)
- [ ] Candle이 막는 것과 허용하는 것 (Q2-2~2-5)
- [ ] OKX 행 9칸의 의미 (02장 표)
- [ ] DTO가 3필드인 이유 (Q3-1)
- [ ] Jackson 3 import 함정 (Q3-6)
- [ ] OKX 업무 오류는 HTTP 200 + code (Q4-1)
- [ ] Builder 주입·생성자 주입의 이득 (Q4-2, 4-3)
- [ ] @Profile로 수동 러너 격리 (Q4-5)
- [ ] 가짜 응답은 진짜처럼 (Q5-1)
- [ ] Testcontainers가 검증하는 것 (Q6-2)
- [ ] CI를 PR·main 둘 다 (Q7-1)
- [ ] Ruleset이 막는 것과 확인 방법 (Q7-4)
- [ ] D를 고른 진짜 이유 (Q8-2, 8-3)

Python	Java / Spring
`@dataclass(frozen=True)`	`record`
`__init__` / `__post_init__`	클래스 이름 생성자 / compact constructor
`raise ValueError(...)`	`throw new IllegalArgumentException(...)`
`x: int` (힌트)	`int x` (강제, 순서 반대)
`int` (무제한)	`int` 약 21억 / `long`
`Decimal` 끼리 `<`	`a.compareTo(b) < 0`
`==` / `is`	`equals` / `==`
`if 1:`	없음 — boolean만
모듈 함수 `_helper()`	`private static` 메서드
`json` + Pydantic 응답 모델	Jackson + DTO record
FastAPI `Depends` / `settings.x`	생성자 주입 / `@Value("${x}")`
`.env.dev` 분리	Spring 프로필 `application-local.yaml`
`logging.getLogger(__name__)`	`LoggerFactory.getLogger(X.class)`
`requirements.txt` / `-dev`	`implementation` / `testImplementation`
`with pytest.raises(E):`	`assertThrows(E.class, () -> ...)`
pytest fixture / `setUp`	`@BeforeEach`
`responses` / `respx`	`MockRestServiceServer`
fixture로 DB 띄우고 정리	Testcontainers + `@ServiceConnection`
`lambda: f()`	`() -> f()`
Alembic	Flyway (실행된 V1은 고치지 않고 V2를 쌓기)
`app/` vs `tests/`	`src/main/java` vs `src/test/java` (경로 맞춤)

다음 단계 — M3

1초봉은 하루 늦을수록 하루치가 영영 사라진다. **upsert + after 페이지네이션 + 단순 속도 제한**으로 백필을 먼저 시작하고, 재시도·재개는 그다음. M3에서는 최소 한 이슈를 이 워크북의 힌트 수준만으로 직접 써 보자.